Forethought: Platform Build Plan

Purpose of this document

This is a comprehensive technical and product specification for building Forethought — a performance-tracked marketplace for economic forecasters and analysts. It is intended to serve as the primary reference document for development, including when working with Claude Code or similar AI-assisted development tools.

The builder has deep domain expertise (Oxford Economics, S&P Global) and will be developing the platform primarily using Claude Code, with support from developer friends as complexity increases. The plan prioritises getting to a functional, near-production-quality product that can be seeded with publicly available forecast data before analyst recruitment begins.

---

1. Platform overview

What Forethought is

Forethought is a platform where economic and macro analysts publish forecasts and research, have their forecast accuracy tracked transparently, and connect with institutional buyers of research and analysis. It combines three functions:

1. **Forecast registry and scoring engine** — analysts submit forecasts on economic variables; the platform tracks accuracy over time using rigorous scoring methods and publishes performance rankings.
2. **Research marketplace** — analysts sell reports, commentary, and ad hoc research services; buyers can evaluate them based on verified performance data.
3. **Consensus data product** — the platform aggregates individual forecasts into consensus and weighted-consensus products that can be sold to institutional clients.

What makes it distinctive

The core differentiator is transparent, standardised performance tracking. Existing players (Oxford Economics, EIU, bank research departments, independent consultancies) sell forecasts and analysis on the strength of brand and credentials. None of them publish systematic records of forecast accuracy. Forethought makes accuracy visible, which shifts the basis of competition from reputation to demonstrated skill.

Seeding strategy

The platform launches pre-populated with publicly available forecasts from the IMF (World Economic Outlook), World Bank (Global Economic Prospects), OECD (Economic Outlook), major central banks, and national statistics offices. These are tracked passively — the institutions don't need to create accounts. This provides an immediate baseline consensus, content for the

public-facing showcase, and a benchmark that early analyst members can position themselves against.

---

2. Technical architecture

2.1 Recommended stack

Framework: Next.js (App Router)

Reasoning: Next.js provides server-side rendering (important for the public showcase and SEO), API routes (eliminating the need for a separate backend initially), and a mature React ecosystem. The App Router architecture supports server components, which improve performance for data-heavy pages like forecast charts and analyst profiles. For a solo builder using Claude Code, having frontend and backend in a single codebase dramatically reduces complexity.

Language: TypeScript throughout

Reasoning: Type safety prevents a large class of bugs, especially important when modelling financial data where a mistyped field can produce subtly wrong calculations. TypeScript also produces better results when working with AI coding assistants because the type definitions serve as implicit documentation.

Database: PostgreSQL

Reasoning: Forecast data is inherently relational — forecasters submit predictions on variables, which are scored against actuals, aggregated into consensus products, and linked to analyst profiles. PostgreSQL handles this naturally. It also supports time-series queries well (important for charting forecast histories), has excellent JSON support (useful for flexible metadata on variables and reports), and scales to the volumes Forethought will need for years. Use a managed PostgreSQL service (Supabase, Neon, or Railway) to avoid database administration overhead.

ORM: Drizzle

Reasoning: Drizzle provides type-safe database queries that integrate well with TypeScript and Next.js. It's lighter-weight than Prisma, produces more predictable SQL (important when you need to optimise time-series queries), and has a simpler migration system. For a platform where the database schema will evolve as features are added, Drizzle's migration workflow is more forgiving.

Authentication: Auth.js (NextAuth v5)

Reasoning: Supports email/password, Google, LinkedIn, and institutional SSO. Free, open-source, and well-integrated with Next.js. LinkedIn auth is particularly relevant — analysts may want to link their Forethought profile to their professional identity, and corporate buyers are likely to authenticate via work accounts.

Payments: Stripe

Reasoning: Industry standard for marketplace payments. Stripe Connect supports the specific model Forethought needs — the platform takes a commission on transactions between analysts and buyers. Stripe also handles subscription billing (for analyst content subscriptions) and one-off payments (for individual report purchases or research briefs).

File storage: S3-compatible object storage (e.g. Cloudflare R2 or AWS S3)

Reasoning: Reports, PDFs, datasets, and other analyst-uploaded content need durable, scalable storage with CDN delivery. R2 is cheaper (no egress fees) and simpler if the rest of the infrastructure isn't heavily AWS-dependent.

Charting: Recharts or D3.js

Reasoning: Forecast data needs to be presented as interactive time-series charts — showing individual analyst forecasts against the consensus, actual outcomes, and historical accuracy. Recharts is simpler for standard chart types; D3 provides more control for custom visualisations like accuracy heatmaps or ranking trajectories. Start with Recharts and move to D3 for specialised views.

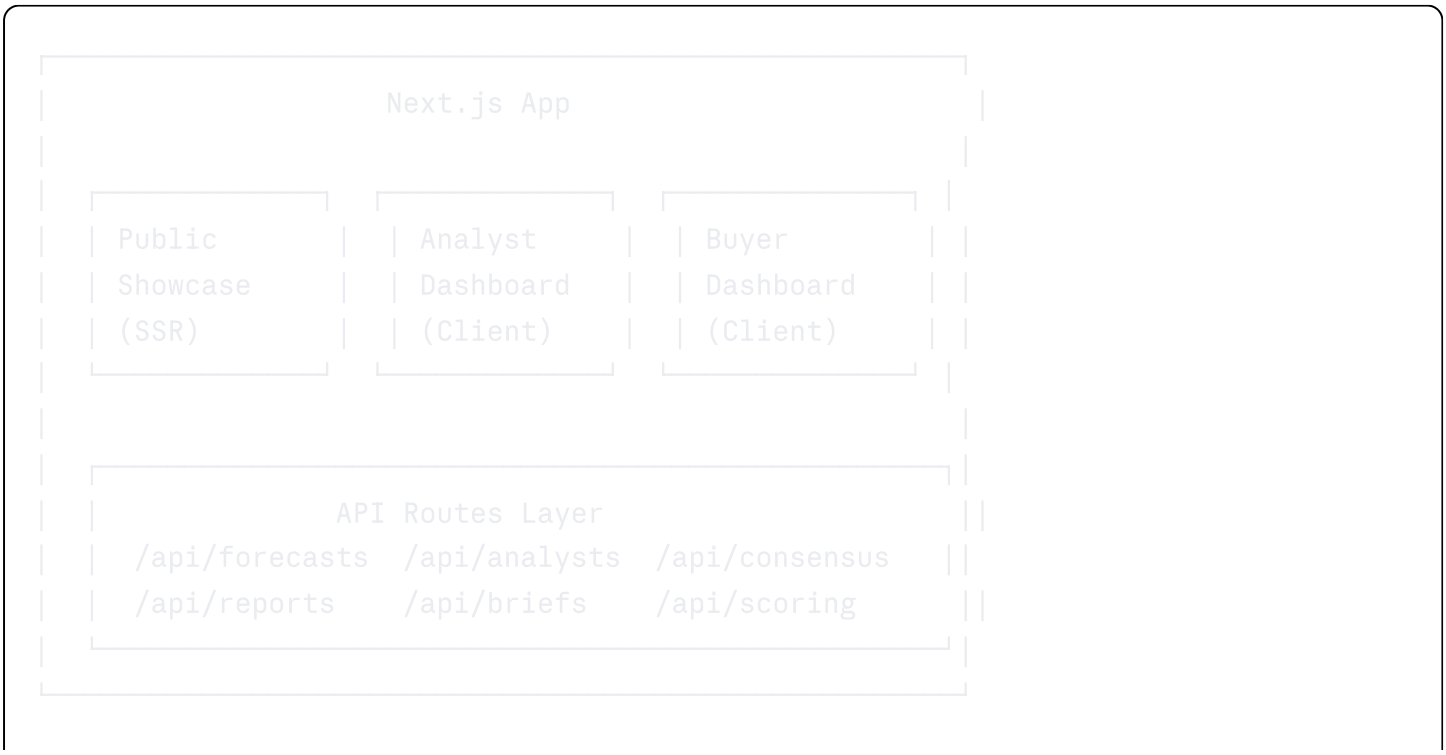
Deployment: Vercel

Reasoning: Native deployment target for Next.js. Handles scaling, CDN, serverless functions, and preview deployments automatically. For a solo builder, the operational simplicity is worth any premium over self-hosting.

Search: Meilisearch or Algolia

Reasoning: Users need to search across analysts, variables, countries, reports, and forecast topics. Full-text search with faceting (filter by country, variable type, time horizon) is essential for the marketplace to function. Meilisearch is open-source and self-hostable; Algolia is managed but costlier. Either works — start with whichever requires less setup time.

2.2 High-level architecture



```
        |              |              |
        ▼              ▼              ▼
┌──────────┐   ┌──────────┐   ┌──────────┐
│PostgreSQL│   │  S3/R2   │   │  Stripe  │
│          │   │ (files)  │   │(payments)│
└──────────┘   └──────────┘   └──────────┘
```

2.3 Core database schema

The schema below covers the essential entities. It will expand as features are added, but this foundation supports all Phase 1 and Phase 2 functionality.

```sql
```

```sql
-- USERS AND ROLES

-- All platform users. The role field determines which dashboard they see.
-- A single user can be both an analyst and a buyer.
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email           TEXT UNIQUE NOT NULL,
    display_name    TEXT NOT NULL,
    bio             TEXT,
    avatar_url      TEXT,
    institution     TEXT,           -- Optional: "IMF", "Oxford Economics", etc.
    is_anonymous    BOOLEAN DEFAULT FALSE,  -- Professionals who want pseudonymous p
    role            TEXT NOT NULL DEFAULT 'viewer',  -- 'viewer', 'analyst', 'buyer'
    verified        BOOLEAN DEFAULT FALSE,  -- Opted into scoring, earned the badge
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- VARIABLES AND FORECASTS

-- The universe of forecastable variables.
-- Each variable is a specific measurable quantity (e.g. "South Africa CPI YoY %")
CREATE TABLE variables (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name            TEXT NOT NULL,           -- "GDP growth rate"
    country         TEXT,                    -- ISO 3166-1 alpha-3, e.g. "ZAF"
    region          TEXT,                    -- "Sub-Saharan Africa", "Global", etc.
    category        TEXT NOT NULL,           -- "macro", "commodity", "political", "
    unit            TEXT NOT NULL,           -- "% YoY", "USD/bbl", "index", "probab
    frequency       TEXT NOT NULL,           -- "quarterly", "annual", "event"
    description     TEXT,
    source_for_actuals TEXT,                 -- Where the actual outcome data comes
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Individual forecast submissions.
-- Each row is one analyst's prediction for one variable at one target period.
CREATE TABLE forecasts (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    analyst_id      UUID NOT NULL REFERENCES users(id),
    variable_id     UUID NOT NULL REFERENCES variables(id),
    target_period   TEXT NOT NULL,           -- "2026-Q3", "2027", "2026-11-03" (ele
    value           DECIMAL NOT NULL,        -- The predicted value
    confidence_low  DECIMAL,                 -- Optional: lower bound of confidence
    confidence_high DECIMAL,                 -- Optional: upper bound of confidence
    submitted_at    TIMESTAMPTZ DEFAULT NOW(),
```

```sql
    is_update       BOOLEAN DEFAULT FALSE,   -- TRUE if this revises a previous fore
    previous_id     UUID REFERENCES forecasts(id),  -- Links to the forecast being r
    source_type     TEXT DEFAULT 'platform', -- 'platform' (submitted by analyst) or
    notes           TEXT                     -- Analyst's reasoning (optional but en
);

-- Actual outcomes, used to score forecasts.
CREATE TABLE actuals (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    variable_id     UUID NOT NULL REFERENCES variables(id),
    target_period   TEXT NOT NULL,
    value           DECIMAL NOT NULL,
    source          TEXT NOT NULL,           -- "IMF WEO April 2027", "Stats SA", et
    published_at    TIMESTAMPTZ NOT NULL,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(variable_id, target_period, source)
);

-- SCORING

-- Computed scores for each forecast once actuals are available.
CREATE TABLE forecast_scores (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    forecast_id     UUID NOT NULL REFERENCES forecasts(id),
    actual_id       UUID NOT NULL REFERENCES actuals(id),
    brier_score     DECIMAL,                 -- For probability-type forecasts
    absolute_error  DECIMAL,                 -- |forecast - actual|
    percentage_error DECIMAL,                -- For continuous variables
    score_vs_consensus DECIMAL,              -- How much better/worse than consensus
    information_contribution DECIMAL,        -- Marginal improvement to weighted con
    scored_at       TIMESTAMPTZ DEFAULT NOW()
);

-- Aggregate analyst performance across variables and time.
-- Recomputed periodically (daily or on new actuals).
CREATE TABLE analyst_rankings (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    analyst_id      UUID NOT NULL REFERENCES users(id),
    variable_id     UUID REFERENCES variables(id),  -- NULL = overall ranking
    category        TEXT,                    -- NULL = overall, or "macro", "commodi
    accuracy_score  DECIMAL NOT NULL,        -- Composite accuracy metric
    consistency     DECIMAL,                 -- How stable their accuracy is over ti
    update_frequency DECIMAL,               -- How actively they revise forecasts
    information_contribution DECIMAL,        -- Average contribution to consensus qu
    rank            INTEGER,
    percentile      DECIMAL,
    period          TEXT NOT NULL,            -- "all_time", "trailing_12m", "traili
```

```sql
    computed_at     TIMESTAMPTZ DEFAULT NOW()
);

-- CONTENT AND MARKETPLACE

-- Reports, articles, and other content published by analysts.
CREATE TABLE content (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    analyst_id      UUID NOT NULL REFERENCES users(id),
    title           TEXT NOT NULL,
    abstract        TEXT,                    -- Public preview
    body            TEXT,                    -- Full content (for on-platform readin
    file_url        TEXT,                    -- S3 link for downloadable reports
    content_type    TEXT NOT NULL,           -- 'report', 'article', 'video', 'podca
    access_level    TEXT DEFAULT 'paid',     -- 'free', 'subscriber', 'paid'
    price_cents     INTEGER,                -- For one-off purchases (NULL = subscri
    currency        TEXT DEFAULT 'USD',
    tags            TEXT[],                  -- For search and discovery
    variables       UUID[],                 -- Links to variables this content discu
    featured        BOOLEAN DEFAULT FALSE,   -- Promoted on the showcase page
    published_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Ad hoc research briefs posted by corporate buyers.
CREATE TABLE research_briefs (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    buyer_id        UUID NOT NULL REFERENCES users(id),
    title           TEXT NOT NULL,
    description     TEXT NOT NULL,
    required_expertise TEXT[],              -- "oil_markets", "south_africa_macro",
    budget_range    TEXT,                    -- "5000-10000 USD" (shown to analysts)
    deadline        DATE,
    status          TEXT DEFAULT 'open',     -- 'open', 'in_progress', 'completed',
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Applications from analysts to research briefs.
CREATE TABLE brief_applications (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    brief_id        UUID NOT NULL REFERENCES research_briefs(id),
    analyst_id      UUID NOT NULL REFERENCES users(id),
    proposal        TEXT NOT NULL,           -- How they'd approach the work
    proposed_fee    INTEGER NOT NULL,        -- In cents
    currency        TEXT DEFAULT 'USD',
    status          TEXT DEFAULT 'pending',  -- 'pending', 'accepted', 'rejected', '
    created_at      TIMESTAMPTZ DEFAULT NOW()
```

```sql
);

-- SUBSCRIPTIONS AND TRANSACTIONS

CREATE TABLE subscriptions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    subscriber_id   UUID NOT NULL REFERENCES users(id),
    analyst_id      UUID NOT NULL REFERENCES users(id),
    tier            TEXT NOT NULL,           -- Defined by the analyst
    price_cents     INTEGER NOT NULL,
    currency        TEXT DEFAULT 'USD',
    stripe_sub_id   TEXT,
    status          TEXT DEFAULT 'active',   -- 'active', 'cancelled', 'expired'
    started_at      TIMESTAMPTZ DEFAULT NOW(),
    expires_at      TIMESTAMPTZ,
    cancelled_at    TIMESTAMPTZ
);

CREATE TABLE transactions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    buyer_id        UUID NOT NULL REFERENCES users(id),
    analyst_id      UUID REFERENCES users(id),  -- NULL for platform purchases (e.g.
    content_id      UUID REFERENCES content(id),
    brief_id        UUID REFERENCES research_briefs(id),
    amount_cents    INTEGER NOT NULL,
    currency        TEXT DEFAULT 'USD',
    platform_fee    INTEGER NOT NULL,        -- Forethought's cut in cents
    stripe_payment_id TEXT,
    type            TEXT NOT NULL,           -- 'content_purchase', 'subscription',
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- CONSENSUS PRODUCTS

-- Computed consensus forecasts (basic and premium/weighted).
CREATE TABLE consensus_forecasts (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    variable_id     UUID NOT NULL REFERENCES variables(id),
    target_period   TEXT NOT NULL,
    consensus_type  TEXT NOT NULL,           -- 'basic' (simple average) or 'weighte
    value           DECIMAL NOT NULL,
    contributor_count INTEGER NOT NULL,
    high            DECIMAL,                 -- Range
    low             DECIMAL,
    computed_at     TIMESTAMPTZ DEFAULT NOW()
);
```

3. Build phases

Phase 1: Forecast Observatory (Months 1–3)

Goal: A publicly accessible platform seeded with institutional forecasts, offering a useful view of macro consensus data that attracts visitors organically and establishes the Forethought brand as a credible source for forecast performance analysis.

What to build:

1.1 Data ingestion pipeline

Build scrapers and manual-import tools for publicly available forecasts:

- IMF World Economic Outlook (April and October releases): GDP growth, inflation, current account, unemployment for all covered countries. Published as downloadable datasets.
- World Bank Global Economic Prospects (January and June): GDP growth, commodity price forecasts.
- OECD Economic Outlook (May and November): GDP, inflation, trade, fiscal balances for OECD and key emerging markets.
- Major central bank forecasts: Fed (Summary of Economic Projections, quarterly), ECB (staff projections, quarterly), Bank of England (Monetary Policy Report, quarterly), SARB (Monetary Policy Review — relevant to your South African knowledge and client base).
- National treasuries and finance departments where publicly available (e.g. SA National Treasury MTBPS and Budget Review).
- Commodity price benchmarks: EIA oil price forecasts, World Bank commodity price forecasts.

For each source, you need:

- A parser that extracts the forecast values and maps them to your variable schema
- A scheduling mechanism (cron job or manual trigger) to ingest new releases
- A source-tracking system so every ingested forecast is traceable to a specific publication

This is the foundation of the platform and worth investing time in getting right. Data quality and traceability are what will differentiate Forethought from someone who just scraped a few PDFs.

Reasoning: This is the single highest-value component for Phase 1. It gives you a populated platform on day one, creates the benchmark layer that early analysts will position against, and produces the raw material for your first data product (consensus forecasts). It also forces you to build the variable taxonomy properly, which is foundational for everything that follows.

1.2 Variable taxonomy

Design and populate the taxonomy of forecastable variables. This needs careful thought because it determines how forecasts are compared, aggregated, and scored.

Structure:

- Country (or "Global")
- Category (macro, commodity, financial, political)
- Variable name (GDP growth, CPI inflation, policy rate, Brent crude, etc.)
- Unit (% YoY, % QoQ, USD/bbl, index level, probability)
- Frequency (quarterly, annual, event-based)
- Target period format (2026-Q3, 2027, specific date for events)

Start with the variables covered by the IMF WEO — that gives you approximately 20 variables across 190+ countries. Expand from there with commodity prices, financial market indicators, and key political events.

Reasoning: The taxonomy must be correct and consistent from the start. If "South Africa GDP growth" is defined differently across two ingested sources (one using calendar year, another using fiscal year), your consensus will be wrong and your scoring will be meaningless. Define the canonical variable first, then map each source's definition to it with any necessary transformations.

1.3 Public showcase (landing page)

Build the public-facing front page. This is not a media site — it's a showcase and conversion tool. It should present:

- **Featured forecasts:** interactive charts showing the current consensus on major variables (global GDP, oil prices, US inflation, key emerging market indicators), with each institutional contributor visible.
- **Performance spotlight:** once enough actuals accumulate, highlight which institutions have been most accurate on specific variables. "Who called the 2026 oil price best?" This is the editorial hook that drives traffic and establishes the brand.
- **Featured analyst content:** once analysts join, their best public-facing pieces are surfaced here. Initially, this section can feature curated summaries of institutional publications.
- **Variable browser:** let visitors explore the full variable universe, see the consensus, and drill into individual contributor forecasts.

Design this to be clean, data-forward, and credible. It should look more like a Bloomberg terminal or Our World in Data than a blog. The aesthetic signals "serious analytical tool" rather than "content platform."

Reasoning: This is the primary acquisition channel. It needs to rank in search (hence SSR), provide immediate value to a visitor who has never heard of Forethought, and create a clear path

toward either (a) signing up as an analyst or (b) expressing interest as a buyer. Every design decision should serve one of these outcomes.

1.4 Scoring engine (initial version)

Build the scoring infrastructure even before analysts join. Apply it to the institutional forecasts you've ingested.

Scoring methodology (drawing on Tetlock's work and standard forecast evaluation):

For continuous variables (GDP, inflation, commodity prices):

- **Mean Absolute Error (MAE)** and **Root Mean Square Error (RMSE)** against actuals
- **Directional accuracy** — did the forecaster get the direction of change right?
- **Score vs. consensus** — was the individual forecast more accurate than the simple average?
- **Forecast revision quality** — did revisions move the forecast closer to the eventual actual? (This rewards good updating behaviour)

For probability-based forecasts (elections, policy decisions, binary events):

- **Brier score** — standard metric for probabilistic forecast accuracy
- **Calibration** — when a forecaster says 70%, does the event occur approximately 70% of the time?
- **Resolution** — does the forecaster distinguish well between events that happen and events that don't?

For all forecast types:

- **Information contribution** — compute the weighted consensus with and without this forecaster; the difference is their marginal contribution to consensus quality. This is a more sophisticated metric to implement but highly valuable; it can be deferred to Phase 2 if needed.

Composite ranking:

- Combine accuracy, consistency (variance of accuracy over time), and update frequency into a single composite score
- Compute for multiple horizons: trailing 3 months, trailing 12 months, all-time
- Compute both overall and per-variable/per-category

Reasoning: Publishing accuracy scores for institutional forecasters (using their publicly available forecasts) is a powerful marketing tool. "We scored the IMF, World Bank, and OECD on their 2024 GDP forecasts — here's who was most accurate" is the kind of analysis that gets shared, cited, and linked to. It also demonstrates the scoring methodology to prospective analyst members before they commit to being scored themselves.

1.5 Basic consensus product

Compute and display consensus forecasts (simple average) across all ingested institutional forecasts. Display on the public showcase. Offer a basic downloadable dataset or API endpoint for anyone who wants the data.

Initially, this can be free or offered at a nominal price. The purpose in Phase 1 is to demonstrate the data product concept and attract users who find it useful. Premium (weighted) consensus comes in Phase 2 once you have enough contributors and scoring data to justify the weighting.

Reasoning: Revenue from consensus data is a medium-term goal, but offering it early — even free — establishes Forethought as a data provider and creates habits among users. It also gives you early feedback on what format and delivery mechanism institutional buyers want (API, Excel download, dashboard, etc.).

Phase 2: Analyst marketplace (Months 3–6)

Goal: Open the platform to independent analysts. Build the tools they need to publish forecasts, sell content, and build verified track records. Launch the matchmaking function for ad hoc research.

2.1 Analyst onboarding and profiles

Build the analyst registration and profile system:

- **Registration flow:** email signup, profile creation (name or pseudonym, bio, areas of expertise, institutional affiliation if any, avatar).
- **Pseudonymous option:** analysts can choose to operate under a pseudonym. Behind the scenes, Forethought verifies their identity (upload of ID or LinkedIn verification), but the public profile shows only the pseudonym. This is critical for professionals who want to participate without risking their employer's disapproval.
- **Variable selection:** during onboarding, analysts choose which variables they want to forecast. They can add more later. No minimum required — they can start with one variable and expand.
- **Initial forecast submission:** to activate their profile, they submit their first forecast(s) on their chosen variable(s). This can be a current-quarter and next-quarter forecast at minimum, up to a three-year quarterly time series if they choose.
- **Profile page:** public-facing page showing the analyst's bio, chosen variables, forecast history (charted), accuracy scores (once available), published content, and verification badge status.

Reasoning: The onboarding flow must be low-friction but structured enough to produce useful data from the first interaction. The pseudonymous option is important — it removes a major barrier for employed professionals who are the most valuable early supply.

2.2 Forecast submission interface

Build the interface for analysts to submit and update forecasts:

- **Submission form:** select variable, select target period, enter forecast value. Optional: confidence interval, brief text note explaining reasoning.
- **Revision workflow:** analysts can update any active forecast at any time. The system records the revision chain (linked via previous_id in the schema). Revision dates and values are visible on the public profile — transparency on how the analyst's view evolved.
- **Quarterly prompt:** the platform sends a quarterly email/notification prompting analysts to review and update their forecasts. They can confirm "no change" with a single click (resubmitting the same value) or update. This satisfies the minimum engagement requirement without forcing artificial updates.
- **Bulk submission:** for analysts covering many variables, provide a CSV/Excel upload option and an API endpoint.
- **Dashboard:** analysts see their own forecasts, how they compare to the current consensus, and (once actuals are available) their accuracy scores.

Reasoning: The forecast submission interface is the core interaction loop of the platform. It must be fast (submitting a forecast should take under 30 seconds), transparent (the analyst sees exactly how their forecast compares to others), and rewarding (they immediately see their position relative to consensus and, over time, their accuracy ranking).

2.3 Content publishing system

Build the tools for analysts to publish and sell content:

- **Content editor:** rich text editor for on-platform articles and reports. Support for charts, tables, images, and embedded forecast data from the platform.
- **File upload:** PDF and other document upload for pre-formatted reports.
- **Access levels:** free (visible on public showcase), subscriber-only, or paid (one-off purchase).
- **Pricing:** analyst sets their own price. Platform displays market data ("reports on Nigerian macro typically sell for $200–$500") to help with price discovery.
- **Preview/abstract:** every piece of content has a public abstract or preview. Buyers can see what they're getting before purchasing.
- **Showcase submission:** analysts can submit content for featuring on the public showcase page. Editorial discretion on what gets featured (initially this is you making the call; later, a combination of algorithmic ranking and editorial judgment).

Reasoning: Content publishing is the Patreon-equivalent function. It gives analysts a reason to join the platform beyond forecast tracking — they can monetise their analysis directly. The

content also populates the showcase, creating the flywheel between analyst activity and platform visibility.

2.4 Verification badge system

Implement the opt-in verification system:

- Analysts who opt into scoring receive a "Verified Forecaster" badge on their profile.
- The badge requires: (a) identity verification (completed during onboarding for pseudonymous users; automatic for named profiles), (b) active forecast submissions on at least one variable for at least one quarter, (c) consent to public accuracy scoring.
- Verified analysts get priority placement in search results, on the showcase, and in matchmaking.
- Unverified analysts can still publish content and sell reports, but their profiles are clearly marked as unverified, and they cannot apply to research briefs (where buyers need to trust performance data).

Reasoning: This creates the market-driven incentive structure discussed earlier. Verification is optional, but the advantages are significant enough that most serious analysts will opt in. Over time, buyers will treat "unverified" as a soft disqualification, which pushes the platform toward universal transparency without mandating it.

2.5 Research brief matchmaking

Build the marketplace for ad hoc research:

- **Brief posting:** corporate buyers post research briefs describing what they need (topic, scope, required expertise, budget range, deadline).
- **Analyst discovery:** briefs are shown to relevant analysts based on their variable coverage and expertise tags. Analysts can also browse open briefs.
- **Application:** analysts submit a short proposal (approach, timeline, fee) in response to a brief.
- **Selection:** the buyer reviews proposals alongside analyst profiles (including accuracy scores for verified analysts) and selects a researcher.
- **Payment:** fee is escrowed via Stripe at selection; released to the analyst (minus platform commission) on completion and buyer approval.
- **Review:** buyers leave a rating after completion. This rating contributes to the analyst's profile alongside their forecast accuracy scores.

Platform commission: 10–15% on matchmaking transactions. This is competitive with GLG and Third Bridge (which charge far higher effective margins) while providing something they don't — transparent performance data to support the buyer's decision.

Reasoning: Matchmaking is the most direct path to revenue. A single research brief at $5,000–$10,000 with a 10–15% commission generates $500–$1,500 for the platform. You need very few of these to cover operating costs in the early stages. It also creates the highest-value interaction for analysts (meaningful paid work) which strengthens retention.

2.6 Subscription system

Build analyst-to-subscriber subscriptions:

- Analysts define tiers (e.g. "Basic: monthly commentary, $20/month" / "Premium: full reports + data access, $100/month").
- Subscribers sign up via Stripe. Annual commitment with one-month cancellation notice (as specified in the original concept).
- Platform takes 8–12% commission (competitive with Patreon's 5–12%, justified by the additional discovery, scoring, and matchmaking infrastructure Forethought provides).
- Subscriber dashboard shows updates from followed analysts, new content, forecast updates.

Reasoning: Subscriptions create recurring revenue for analysts (improving retention) and for the platform (via commission). But this is secondary to matchmaking as a revenue driver in the early stages — don't over-invest in subscription infrastructure before you have enough analysts to make it worthwhile.

2.7 Weighted consensus product (premium)

Once you have 10+ active forecasters on a given variable (including institutional benchmarks), compute and offer the weighted consensus:

- Weight each forecaster's contribution by their historical accuracy on that variable (and optionally related variables).
- Update weights as new actuals come in and scores are recomputed.
- Offer the weighted consensus as a premium data product (subscription via Stripe, or per-download).
- Clearly differentiate from the basic (free or cheap) consensus: show that the weighted version outperforms the simple average (backtest this using historical data from the ingested institutional forecasts).

Reasoning: This is the data moat. The weighted consensus improves as the platform grows (more contributors, more actuals, better-calibrated weights). It's a product that no single analyst or institution can replicate, because it requires the scoring infrastructure and the panel of contributors. Pricing should start modest ($500–$2,000/year for a single-variable subscription, with institutional bundles higher) and increase as the track record demonstrates value.

Phase 3: Scale and deepen (Months 6–12)

Goal: Expand coverage, improve the data products, build institutional relationships, and refine the platform based on early user feedback.

3.1 API for data products

Build a REST API (and optionally a lightweight Python/R client library) for institutional buyers to programmatically access consensus forecasts, individual analyst forecasts (where public), and scoring data.

Endpoints:

- `GET /api/v1/consensus/{variable_id}` — current consensus (basic or weighted)
- `GET /api/v1/forecasts/{variable_id}` — all individual forecasts for a variable
- `GET /api/v1/analysts/{analyst_id}/performance` — scoring data for an analyst
- `GET /api/v1/rankings` — current rankings by category, variable, or overall
- `GET /api/v1/variables` — browse the variable taxonomy

Authentication via API keys. Rate-limited free tier for exploration; paid tier for production use.

Reasoning: Institutional buyers (asset managers, trading houses, corporate strategy teams) want data feeds, not websites. An API transforms Forethought from a tool they visit into infrastructure they integrate. This is also a prerequisite for selling the consensus product at institutional price points.

3.2 Information contribution metric

Implement the full information-contribution scoring:

- For each forecast submission, compute: what was the weighted consensus with this forecast included vs. excluded?
- The difference in accuracy (measured against actuals) is the analyst's information contribution.
- Display on analyst profiles: "This analyst's forecasts have improved the weighted consensus for South African GDP by X% on average."

Reasoning: This is the metric that matters most for the premium consensus product. It tells buyers *which analysts are actually making the consensus better*, as opposed to just submitting forecasts that converge with everyone else's. It also provides a more nuanced view of analyst value than raw accuracy alone — an analyst might not have the best individual accuracy but could still be the most valuable contributor to the consensus because their errors are uncorrelated with others'.

3.3 Institutional outreach

This is a business development phase, not a technical one, but the platform needs to support it:

- Build an institutional profile type: allows organisations (OE, EIU, central banks) to have a branded presence with multiple analysts under one umbrella.
- Build a reporting dashboard for institutional clients: show them what they're getting from their consensus subscription, which analysts contributed, how the consensus has evolved.
- Create collateral: "Forethought Forecast Performance Report" — a periodic publication scoring institutional forecasters against actuals. This is both a marketing tool and a demonstration of the platform's methodology.

3.4 Expanded variable coverage

Based on user demand and analyst supply, expand into:

- Financial market variables (equity indices, bond yields, exchange rates)
- Commodity-specific forecasts (beyond oil: metals, agriculture)
- Political events and elections
- Country-specific deep dives (e.g. comprehensive South African macro coverage, leveraging your network)

3.5 Mobile optimisation

Ensure the platform works well on mobile for both analysts (submitting quick forecast updates) and buyers (browsing analyst profiles, reading content). A progressive web app (PWA) is sufficient — native mobile apps are unnecessary at this scale.

---

4. Key design principles

4.1 Data integrity above all

Every forecast, every actual, every score must be traceable and immutable. Analysts cannot delete or retroactively modify submitted forecasts. Actuals are sourced and cited. Scoring methodology is public and versioned. If you change the scoring formula, old scores are preserved under the old methodology and new scores are clearly labelled. Trust in the data is Forethought's entire value proposition — any compromise here is existential.

4.2 Analyst experience drives supply

If the platform is annoying, slow, or unrewarding for analysts, they won't submit forecasts and they won't publish content. Every interaction — from submitting a forecast to checking their ranking to uploading a report — should be fast, clear, and satisfying. Show them their impact: "Your latest forecast moved the consensus by 0.3 percentage points." Show them their trajectory: "Your accuracy on oil prices has improved from the 40th to the 65th percentile over six months."

4.3 Public showcase drives demand

The showcase must be genuinely useful to a visitor who never creates an account. It should

answer the question: "What does the market think about X?" If someone googles "South Africa GDP forecast 2027" and your showcase page ranks, that's a potential buyer or analyst seeing Forethought for the first time. Optimise for this.

4.4 Revenue follows value

Don't paywall aggressively in Phase 1. The basic consensus, the showcase, and the institutional scoring reports should be free or nearly free. Revenue comes from matchmaking commissions, premium consensus subscriptions, and analyst subscription commissions — all of which require a functioning marketplace with both supply and demand. Premature monetisation of the data layer will slow growth.

---

5. Technical notes for Claude Code

5.1 Project structure

```
forethought/
├── src/
│   ├── app/                    # Next.js App Router
│   │   ├── (public)/           # Public showcase pages (SSR)
│   │   │   ├── page.tsx        # Landing/showcase page
│   │   │   ├── variables/      # Variable browser
│   │   │   ├── analysts/       # Public analyst directory
│   │   │   └── rankings/       # Performance leaderboards
│   │   ├── (dashboard)/        # Authenticated dashboard
│   │   │   ├── analyst/        # Analyst dashboard
│   │   │   ├── buyer/          # Buyer dashboard
│   │   │   └── admin/          # Admin tools
│   │   ├── api/                # API routes
│   │   │   ├── forecasts/
│   │   │   ├── analysts/
│   │   │   ├── consensus/
│   │   │   ├── content/
│   │   │   ├── briefs/
│   │   │   ├── scoring/
│   │   │   └── auth/
│   │   └── layout.tsx
│   ├── components/             # Shared UI components
│   │   ├── charts/             # Forecast charts, accuracy visualisations
│   │   ├── ui/                 # Design system components
│   │   └── forms/              # Forecast submission, content upload, etc.
│   ├── lib/
│   │   ├── db/                 # Drizzle schema, migrations, queries
│   │   ├── scoring/            # Scoring engine (Brier, MAE, RMSE, etc.)
│   │   ├── consensus/          # Consensus computation (basic + weighted)
```

```
│   │   ├── ingestion/          # Data scrapers and import tools
│   │   ├── auth/               # Auth.js configuration
│   │   ├── payments/           # Stripe integration
│   │   └── storage/            # S3/R2 file handling
│   └── types/                  # TypeScript type definitions
├── scripts/
│   ├── ingest/                 # One-off and scheduled data import scripts
│   │   ├── imf-weo.ts
│   │   ├── world-bank.ts
│   │   ├── oecd.ts
│   │   └── central-banks/
│   └── scoring/                # Batch scoring jobs
├── drizzle/                    # Database migrations
├── public/                     # Static assets
├── .env.local                  # Environment variables
├── next.config.ts
├── drizzle.config.ts
├── tsconfig.json
└── package.json
```

5.2 Environment variables

```env
```

```
# Database
DATABASE_URL=postgresql://...

# Auth
AUTH_SECRET=...
AUTH_GOOGLE_ID=...
AUTH_GOOGLE_SECRET=...
AUTH_LINKEDIN_ID=...
AUTH_LINKEDIN_SECRET=...

# Stripe
STRIPE_SECRET_KEY=...
STRIPE_PUBLISHABLE_KEY=...
STRIPE_WEBHOOK_SECRET=...

# Storage
S3_BUCKET=...
S3_REGION=...
S3_ACCESS_KEY=...
S3_SECRET_KEY=...
S3_ENDPOINT=...  # For R2 or other S3-compatible

# Search
MEILISEARCH_HOST=...
MEILISEARCH_API_KEY=...
```

5.3 Key dependencies

```
json
```

```
{
  "dependencies": {
    "next": "^15",
    "react": "^19",
    "typescript": "^5",
    "drizzle-orm": "latest",
    "drizzle-kit": "latest",
    "@auth/core": "latest",
    "@auth/drizzle-adapter": "latest",
    "next-auth": "latest",
    "stripe": "latest",
    "@stripe/stripe-js": "latest",
    "recharts": "latest",
    "@aws-sdk/client-s3": "latest",
    "meilisearch": "latest",
    "zod": "latest",
    "date-fns": "latest"
  }
}
```

5.4 Coding standards

- All database queries through Drizzle (no raw SQL except for complex analytical queries in the scoring engine)
- All API inputs validated with Zod schemas
- All financial amounts stored as integers (cents) to avoid floating-point issues
- All timestamps in UTC
- All forecast values stored as DECIMAL, not FLOAT, for precision
- Server components by default; client components only when interactivity is required
- Error handling: every API route returns structured error responses with appropriate HTTP status codes
- Logging: structured JSON logs for all forecast submissions, scoring events, and transactions

5.5 Build sequence for Claude Code

When working with Claude Code, build in this order:

1. **Database schema and migrations** — set up Drizzle, create all tables, run migrations
2. **Variable taxonomy seed** — create the initial set of variables matching IMF WEO coverage
3. **Data ingestion scripts** — IMF WEO first (it's the most structured), then expand
4. **API routes for reading** — GET endpoints for variables, forecasts, consensus

5. **Public showcase pages** — landing page, variable browser, basic charts
6. **Consensus computation** — basic average consensus, displayed on the showcase
7. **Scoring engine** — compute accuracy metrics for ingested institutional forecasts
8. **Authentication** — user registration, login, role assignment
9. **Analyst dashboard** — forecast submission, profile management
10. **Content publishing** — upload, paywall, showcase submission
11. **Verification badge** — opt-in scoring, badge display
12. **Research briefs** — posting, application, selection flow
13. **Stripe integration** — payments for content, subscriptions, matchmaking
14. **Weighted consensus** — performance-weighted aggregation
15. **Rankings and leaderboards** — public performance tables

Each step should be a working increment — deploy after each one and verify it works in production before moving to the next.

---

6. Risks and mitigations

Risk: Data ingestion is harder than expected. Institutional forecasts are published in inconsistent formats (PDFs, Excel files, web pages, APIs). Each source requires custom parsing logic, and formats change without notice.

Mitigation: Start with the most structured sources (IMF WEO publishes machine-readable datasets). Accept that some sources will require manual data entry initially. Build alerting so you know when an ingestion script breaks due to a format change.

Risk: Analysts don't join. The platform launches with institutional data but no independent analysts.

Mitigation: Use your OE and S&P network to recruit the first 10–20 analysts personally. Offer the first cohort zero platform commission for their first year. Their early participation seeds the marketplace and generates the social proof needed to attract others.

Risk: Buyers don't post research briefs. The matchmaking function requires corporate demand.

Mitigation: Again, your network is the asset here. Identify 5–10 potential corporate clients from your professional contacts and approach them directly. Offer the first few matchmaking engagements commission-free to demonstrate the model. One successful engagement creates a case study that sells the next ten.

Risk: Scoring methodology is contested. Analysts or institutions disagree with how they're being scored.

Mitigation: Publish the methodology in full, including the code. Invite feedback. Be transparent about the limitations (e.g. "scoring on annual GDP forecasts requires waiting for revised actuals,

which can take 12–18 months"). Versioning the methodology means you can improve it without invalidating historical scores.

Risk: Legal challenges from institutions being scored without consent. An institution could argue that publicly scoring their forecasts constitutes misuse of their data.

Mitigation: The forecasts being scored are publicly published by the institutions themselves. Scoring them is commentary and analysis of public information — well within fair use and freedom of expression protections. However, consult a lawyer before launch to confirm this for the jurisdictions you're operating in. Don't reproduce the forecasts themselves in bulk; present scores and comparative analysis.

---

7. Success metrics

Phase 1 (months 1–3):

- Platform live with 50+ variables populated with institutional forecast data
- At least 2 scoring reports published (e.g. "Who forecast 2025 GDP most accurately?")
- 500+ monthly visitors to the showcase
- At least 3 analysts expressing interest in joining

Phase 2 (months 3–6):

- 15+ active independent analysts submitting forecasts
- 5+ pieces of paid content published on the platform
- First research brief posted and matched
- First revenue (any amount)

Phase 3 (months 6–12):

- 50+ active analysts
- 10+ corporate buyers or subscribers
- Weighted consensus product launched for at least 5 variables
- Monthly revenue covering basic operating costs (hosting, Stripe fees)
- At least one institutional forecaster (research house, consultancy, or bank research team) with a profile on the platform